

A Practical Course on Mathematical Modelling

Lecture 2 - Fundamentals of Julia language

Prof. Viet Hung Nguyen

Clermont Auvergne University - Clermont-Ferrand - France

The birth of Julia

- Created in 2009 by Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah in MIT
- Publicly announced in 2012
- Designed for scientific computing, numerical optimization, and data analysis
- Main goal: combine Python's ease of use with C-like performance

The two-language problem

Julia aims to eliminate the need to prototype algorithms in a high-level language and rewrite them in a low-level language to obtain high performance.

Main features and comparisons with C and Python

Feature	Julia	Python	C
Execution	JIT-compiled	Interpreted	Compiled
Performance	Close to C	Usually slower	Very high
Syntax	High-level	High-level	Low-level
Typing	Dynamic	Dynamic	Static
Memory	Automatic	Automatic	Mostly manual
Multiple dispatch	Native	Limited	No
Interactive use	Excellent	Excellent	Limited

Julia for Scientific Computing

- Near-C performance with high-level, readable syntax
- Efficient numerical loops without mandatory vectorization
- Native support for matrices, complex numbers, and mathematical notation
- Multiple dispatch for generic and reusable algorithms
- Built-in support for parallel and distributed computing
- Rich scientific ecosystem:
 - `LinearAlgebra` for linear algebra
 - `DifferentialEquations.jl` for differential equations
 - `Plots.jl` for data visualization

Main strength

Julia allows scientists to write readable mathematical code without sacrificing computational performance.

Julia for Mathematical Optimization

- JuMP: an expressive algebraic modeling language
- Support for a wide range of optimization problems:
 - linear and mixed-integer optimization
 - nonlinear and conic optimization
 - stochastic and multiobjective optimization
- Common interface to many solvers: HiGHS, Gurobi, CPLEX, Ipopt, and SCIP
- Easy access to variables, constraints, dual values, and solver statistics
- Models and specialized algorithms can be implemented in the same high-performance language

Main strength

Julia combines mathematical modeling, algorithm development, and high-performance implementation.

Julia and JuMP: Support and Resources

- **Official websites**

- Julia language: <https://julialang.org>
- JuMP: <https://jump.dev>

- **Documentation and tutorials**

- Julia manual and learning resources: <https://docs.julialang.org>
- JuMP tutorials with executable examples:
<https://jump.dev/JuMP.jl/stable/tutorials>

- **Community support**

- Julia Discourse forum: <https://discourse.julialang.org>
- Questions and examples on Stack Overflow
- Active open-source development on GitHub

- **Cross-platform availability**

- Available on Windows, macOS, and Linux
- Consistent package management across platforms
- Usable through Pluto, Jupyter, and the command line

A mature and accessible ecosystem

Julia and JuMP provide extensive documentation, practical tutorials, and active community support for both beginners and advanced users.

Installing Julia

- 1 Go to <https://julialang.org/downloads>
- 2 Download the installer for:
 - Windows
 - macOS
 - Linux
- 3 Follow the installation instructions
- 4 Start Julia to open the interactive prompt

Cross-platform installation

Julia uses the same language and package manager on Windows, macOS, and Linux.

Installing Essential Packages

From the Julia prompt, enter:

```
using Pkg
```

```
Pkg.add("Pluto")
```

```
Pkg.add("Plots")
```

```
Pkg.add("JuMP")
```

```
Pkg.add("HiGHS")
```

Pluto Interactive notebooks

Plots Data visualization

JuMP Optimization modeling

HiGHS Open-source optimization solver

Important

A package is installed only once, but must be loaded with `using` in each new Julia session.

- **Command line**

- Julia's interactive prompt (REPL)
- Convenient for testing short instructions

- **Integrated development environments**

- Visual Studio Code with the Julia extension
- Recommended for developing larger projects

- **Interactive notebooks**

- Jupyter notebooks through IJulia.jl
- **Pluto.jl**: reactive notebooks designed for Julia

Why Pluto?

Pluto automatically updates dependent cells when a variable changes. It is particularly suitable for interactive teaching, experimentation, and reproducible scientific computing.

The Julia REPL and Its Different Modes

The **REPL** allows instructions to be executed interactively.

Prompt	Mode	Activation	Example
julia>	Julia	Default	<code>sqrt(16)</code>
help?>	Help	Type ?	<code>println</code>
shell>	Shell	Type ;	<code>pwd</code>
pkg>	Package	Type]	<code>add Plots</code>

- Press Enter to execute an instruction.
- Press Backspace on an empty line to return to Julia mode.
- Press Ctrl+D to leave the REPL.

Interactive workflow

The REPL is useful for testing expressions, consulting documentation, managing packages, and running system commands.

Examples of Julia Instructions in REPL Mode

Calculations and variables

```
julia> 2 + 3 * 4  
14
```

```
julia> x = 5  
5
```

```
julia> x^2 + 2*x + 1  
36
```

Built-in functions

```
julia> sqrt(16)  
4.0
```

```
julia> sin(pi / 2)  
1.0
```

Vectors and matrices

```
julia> v = [1, 2, 3]  
3-element Vector{Int64}
```

```
julia> sum(v)  
6
```

```
julia> A = [1 2; 3 4]  
2x2 Matrix{Int64}
```

Loading a package

```
julia> using LinearAlgebra
```

```
julia> det(A)  
-2.0
```

Useful Julia REPL Shortcuts

Shortcut	Action
Up / Down	Browse the command history
Tab	Complete a command or variable name
Ctrl+C	Interrupt the current computation
Ctrl+L	Clear the REPL screen
Ctrl+D	Exit Julia
?	Enter Help mode
]	Enter Package mode
;	Enter Shell mode

Returning to Julia mode

Press Backspace on an empty line to leave Help, Package, or Shell mode.

Main Structure of a Julia Program

```
# 1. Load packages
using Statistics

# 2. Define constants
const SCALE = 2.0

# 3. Define functions
function transform(data)
    return SCALE .* data
end

# 4. Main program
function main()
    data = [1, 2, 3]
    result = transform(data)
    println("Result: ", result)
end

# 5. Start the program
main()
```

A Julia program usually contains:

- 1 package imports;
- 2 constants and data;
- 3 function definitions;
- 4 a main function;
- 5 a call to the main function.

Good practice

Place computations inside functions to improve code organization, reusability, and performance.

Executing a Julia Program from the REPL

Suppose that the program is stored in `test.jl`.

Execute the complete file:

```
julia> include("test.jl")
```

Execute a file located in another directory:

```
julia> include("/path/to/test.jl")
```

Check or change the current directory:

```
julia> pwd()  
julia> cd("/path/to/project")  
julia> include("test.jl")
```

Important

`include` executes the file in the current Julia session: the defined variables and functions remain available in the REPL.

Installing Julia in Visual Studio Code

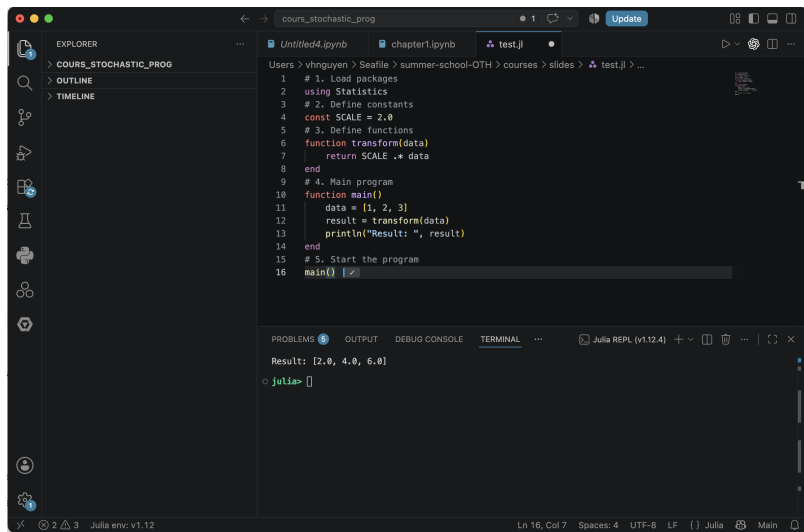
- 1 Install Julia from <https://julialang.org/downloads>
- 2 Install Visual Studio Code from <https://code.visualstudio.com>
- 3 Open the **Extensions** panel in VS Code:
 - press Ctrl+Shift+X;
 - search for Julia;
 - install the official Julia extension.
- 4 Create or open a file with the extension `.jl`
- 5 Start the Julia REPL from the Command Palette:

Julia: Start REPL

Running Julia code

Select some code and press Shift+Enter to execute it in the integrated Julia REPL.

Running Julia in Visual Studio Code



The screenshot shows the Visual Studio Code interface with a dark theme. The Explorer sidebar on the left shows a project named 'COURS_STOCHASTIC_PROG' with files 'Untitled4.ipynb', 'chapter1.ipynb', and 'test.jl'. The main editor displays the contents of 'test.jl', which is a Julia script. The script includes comments for each step, package loading, constant definition, function definition, and a main program. The terminal at the bottom shows the output of the script, which is 'Result: [2.0, 4.0, 6.0]'. The status bar at the bottom indicates 'Julia env: v1.12' and 'Ln 16, Col 7'.

```
1 # 1. Load packages
2 using Statistics
3 # 2. Define constants
4 const SCALE = 2.0
5 # 3. Define functions
6 function transform(data)
7     return SCALE .* data
8 end
9 # 4. Main program
10 function main()
11     data = [1, 2, 3]
12     result = transform(data)
13     println("Result: ", result)
14 end
15 # 5. Start the program
16 main()
```

Result: [2.0, 4.0, 6.0]

julia>

The Julia extension provides an integrated REPL. Select an instruction and press **Shift+Enter** to execute it.

Installing and Starting Pluto

1. Install Pluto from the Julia REPL

```
julia> using Pkg  
julia> Pkg.add("Pluto")
```

2. Start Pluto

```
julia> using Pluto  
julia> Pluto.run()
```

Pluto opens automatically in the default web browser.

Next sessions

Pluto is installed only once. To start it again, simply use:

```
using Pluto    Pluto.run()
```

Using a Pluto Notebook

- 1 Create a new notebook or open an existing `.jl` notebook.
- 2 Enter Julia instructions in a cell.
- 3 Click the **Run** button or press `Shift+Enter`.
- 4 Add new cells using the `+` buttons.
- 5 Save the notebook as a Julia `.jl` file.

Reactive execution

When a variable is modified, Pluto automatically re-executes all dependent cells and updates their results.

Implementing the Program in Pluto

Cell 1: Load the package

```
using Statistics
```

Cell 2: Define the parameter

```
scale = 2.0
```

Cell 3: Define the function

```
function transform(data)
    return scale .* data
end
```

Cell 4: Perform the computation

```
data = [1, 2, 3]
result = transform(data)
```

Cell 5: Display the result

```
println("Result: ",
        result)
```

Reactive execution

If scale or data is modified, Pluto automatically re-executes the dependent cells and updates result.

Pluto principle

Each global variable should normally be defined in only one cell.

The Pluto Notebook Interface



Pluto.jl  Sauvegarder le notebook...

```
using Statistics ✓  
  
scale = 2.0  
scale=2.0  
  
Transform (generic function with 1 method)  
function Transform(data)  
    return scale .* data  
end  
  
[2.0, 4.0, 6.0]  
begin  
    data=[1,2,3]  
    result=Transform(data)  
end  
  
println("Result =",result)  
Result = [2.0, 4.0, 6.0]
```

- Julia instructions are entered into interactive cells.
- Results appear immediately below each cell.
- The + buttons create new cells.
- Cells are automatically re-evaluated when their dependencies change.

Reactive execution

Changing one variable automatically updates all dependent results.

Assigning values to variables

In Julia, a variable is created by assigning a value with `=`.

Numbers and Boolean values

```
zipcode = 44000  
price = 29.95  
positive = true
```

Characters and strings

```
dot = '.'  
sentence = "."
```

Julia infers the type from the assigned value: for example, 44000 is an integer literal and 29.95 is a floating-point literal.

Variable names and Unicode

Julia supports Unicode characters in identifiers. This is particularly useful for mathematical notation.

```
delta = 0.1  
# In the Julia REPL, type \Delta then press TAB:  
Delta = 0.1      # displayed as the Greek capital delta
```

Many Unicode symbols can be entered in the Julia REPL by typing a LaTeX-like command followed by TAB.

Examples include Greek letters and even emoji. For readable scientific code, prefer names whose meaning remains clear to the reader.

Inspecting the type of an expression

Use `typeof(...)` to obtain the type of a value or expression.

```
julia> zipcode = 44000  
44000
```

```
julia> typeof(zipcode)  
Int64
```

```
julia> typeof(29.95)  
Float64
```

The exact default integer type depends on the architecture; on a typical 64-bit machine, integer literals such as 44000 are `Int64`.

A partial hierarchy of numeric types

Parent type	Subtype	Examples
Number	Complex	complex numbers
Number	Real	real-valued numbers
Real	AbstractFloat	Float16, Float32, Float64
Real	Integer	Bool, Signed, Unsigned
Signed	integer types	Int8, Int16, Int32, Int64, Int128
Unsigned	integer types	UInt8, UInt16, UInt32, UInt64, UInt128
Real	Rational	exact rational numbers

For a type `T`, `supertype(T)` returns its parent type and `subtypes(T)` lists its direct child types.

Integer and floating-point representations

Useful functions for inspecting a numeric type T:

Function	Meaning
<code>sizeof(T)</code>	number of bytes used by a value of type T
<code>typemin(T)</code>	smallest finite value representable by T
<code>typemax(T)</code>	largest finite value representable by T

```
julia> sizeof(Int16)
```

```
2
```

```
julia> typemin(Int16), typemax(Int16)  
(-32768, 32767)
```

The keyword `const` declares that a global binding should not be reassigned to a value of a different type.

```
const g = 9.81
```

Julia also provides useful predefined mathematical values:

- `pi` or π : the constant π ;
- `Inf`: positive floating-point infinity;
- `NaN`: “not a number.”

Displaying values with print and println

`print` displays output without automatically adding a newline; `println` adds one.

```
print(price)
println()
println(zipcode)
```

```
println("Zip Code: ", zipcode)
println(zipcode, " | ", price, " EUR ", dot)
println("Zip Code: $zipcode")
```

The last line illustrates **string interpolation**: `$zipcode` is replaced by the value of the variable.

Formatted output with @printf

The Printf standard library provides C-style formatted output.

using Printf

```
@printf("Zip Code: %d\n", zipcode)
```

```
@printf("pi = %.2f\n", pi)
```

Common format specifiers include %d for integers and %f for floating-point values. The precision can be controlled, for example with %.2f.

Quick inspection with @show

@show displays both an expression and its value. It is convenient for quick checks while developing code.

```
julia> zipcode = 44000
```

```
julia> @show zipcode  
zipcode = 44000
```

Unlike `println(zipcode)`, the output explicitly identifies the expression being evaluated.

Reading input from the user

`getline()` reads one line from standard input and returns a `String`.

```
name = getline()  
typeof(name)           # String
```

To read a number, parse the string into the required numeric type:

```
num = parse{Int64, getline()}  
typeof(num)             # Int64
```

Reading several lines

`readlines()` reads all remaining lines and stores them in a one-dimensional array of strings.

```
lines = readlines()  
typeof(lines)      # Vector{String}
```

In an interactive terminal, each line is delimited by ENTER; input is terminated by an end-of-file signal (commonly `Ctrl-D` on Unix-like systems).

For simple interactive terminal menus, Julia also provides `REPL.TerminalMenus`.

Comments document code and are ignored during execution.

```
# This is a one-line comment
```

```
#=
```

```
This is a  
multiline comment.
```

```
=#
```

Use comments to explain *why* a non-obvious operation is performed, rather than simply repeating what the code already says.

Arithmetic operators

Let a and b be numeric variables.

Expression	Operation	Expression	Operation
$-a$	unary minus	$a + b$	addition
$a - b$	subtraction	$a * b$	multiplication
a / b	division	$a ^ b$	power
$a \div b$	integer division	$a \% b$	remainder

In Julia, the Unicode operator $\div$ can be entered as `\div` followed by TAB.

Updating operators

Julia provides concise forms for updating a variable.

Short form	Equivalent form	Operation
<code>a += b</code>	<code>a = a + b</code>	addition
<code>a -= b</code>	<code>a = a - b</code>	subtraction
<code>a *= b</code>	<code>a = a * b</code>	multiplication
<code>a /= b</code>	<code>a = a / b</code>	division
<code>a ^= b</code>	<code>a = a ^ b</code>	power
<code>a %= b</code>	<code>a = a % b</code>	remainder

Numeric comparisons

Comparison expressions return a Boolean value: `true` or `false`.

Operator	Meaning	Operator	Meaning
<code>==</code>	equal to	<code>!=</code> or <code>≠</code>	not equal to
<code><</code>	less than	<code><=</code> or <code>≤</code>	less than or equal to
<code>></code>	greater than	<code>>=</code> or <code>≥</code>	greater than or equal to

Unicode comparison operators such as `≤`, `≥`, and `≠` are valid Julia syntax.

Common arithmetic functions

Function	Meaning	Function	Meaning
<code>div(x,y)</code>	truncated division	<code>mod(x,y)</code>	modulus
<code>abs(x)</code>	absolute value	<code>sign(x)</code>	sign: $-1, 0, +1$
<code>sqrt(x)</code>	square root	<code>exp(x)</code>	e^x
<code>exp2(x)</code>	2^x	<code>log(x)</code>	natural logarithm
<code>log(b,x)</code>	base- b logarithm	<code>factorial(n)</code>	$n!$
<code>binomial(n,p)</code>	$\binom{n}{p}$		

Rounding functions

Let x be a number and T a numeric type.

Function	Operation	Typical return type
<code>round(x)</code>	nearest integer-valued number	<code>typeof(x)</code>
<code>round(T,x)</code>	nearest integer, converted to T	T
<code>round(x,digits=n)</code>	round to n decimal digits	<code>typeof(x)</code>
<code>floor(x)</code>	round toward $-\infty$	<code>typeof(x)</code>
<code>ceil(x)</code>	round toward $+\infty$	<code>typeof(x)</code>
<code>trunc(x)</code>	round toward zero	<code>typeof(x)</code>

The typed variants `floor(T,x)`, `ceil(T,x)`, and `trunc(T,x)` return a value of type T .

Trigonometric functions

Function	Meaning
<code>sin(x)</code> , <code>cos(x)</code> , <code>tan(x)</code>	angle x in radians
<code>sind(x)</code> , <code>cosd(x)</code> , <code>tand(x)</code>	angle x in degrees
<code>deg2rad(x)</code>	convert degrees to radians
<code>rad2deg(x)</code>	convert radians to degrees

Useful functions for strings and characters

Function	Meaning
<code>reverse(s)</code>	reverse string s
<code>occursin(c,s)</code>	test whether c occurs in s
<code>split(s,c)</code>	split s at occurrences of c
<code>endswith(s,s2)</code>	test whether s ends with s2
<code>uppercase(s)</code>	convert letters in s to uppercase
<code>textwidth(s)</code>	number of columns needed to display s
<code>isdigit(c)</code>	test whether c is a decimal digit
<code>isletter(c)</code>	test whether c is a letter

Use `if` when code should run conditionally

The body of an `if` statement is executed only when its condition evaluates to true.

```
if zipcode == 4020  
  println("Welcome to Linz")  
end
```

The keyword `end` closes the conditional block.

Use else to handle the alternative case

```
if zipcode == 4020
  println("Welcome to Linz")
else
  println("Welcome to Austria")
end
```

Exactly one of the two branches is executed.

Short alternatives: `ifelse` and the ternary operator

For simple value selection, Julia offers compact expressions.

```
place = ifelse(zipcode == 4020, "Linz", "Austria")
```

```
place = zipcode == 4020 ? "Linz" : "Austria"
```

- The ternary operator evaluates only the selected branch.
- `ifelse` is a function and evaluates both value arguments before the call.

For blocks containing several instructions, ordinary `if/else` remains clearer.

Use elseif for several mutually exclusive cases

```
if zipcode == 4020
  println("Welcome to Linz Central area")
elseif zipcode == 4030
  println("Welcome to Linz South area")
elseif zipcode == 4040
  println("Welcome to Linz North area")
else
  println("Welcome to Austria")
end
```

Conditions are tested from top to bottom; execution stops at the first true branch.

Combine conditions with logical operators

Let *a* and *b* be Boolean conditions.

Operator	Expression	Meaning
!	!a	NOT a
&&	a && b	a AND b
	a b	a OR b

```
countrycode == "AT" && zipcode == 4020
```

`&&` and `||` use short-circuit evaluation: the second condition is evaluated only when needed.

A while loop repeats while a condition is true

```
zipcode = 4020

while zipcode <= 4040
    print(zipcode, " ")
    zipcode = zipcode + 10
end
```

Output:

4020 4030 4040

The loop body must eventually make the condition false unless an infinite loop is intended.

A for loop iterates over a collection

General form:

```
for variable in collection
    # instructions
end
```

Typical iterable collections include ranges, strings, tuples, arrays, sets, and dictionaries.

```
for i in 1:10
    print(i, " ")
end
```

Ranges make numerical loops concise

```
for i in 1:10
    print(i, " ")
end
```

```
for i in 1:2:10
    print(i, " ")
end
```

The range `start:step:stop` controls both the increment and the final bound.

The same loop syntax works with many collections

```
for c in "Linz"  
    print(c, " ")  
end
```

```
for zc in (4020, 4030, 4040)  
    print(zc, " ")  
end
```

```
for zc in [4020, 4030, 4040]  
    print(zc, " ")  
end
```

The loop consumes the collection through Julia's iteration interface; it does not need to know the concrete data structure in advance.

Iterating over sets and dictionaries

```
for zc in Set([4020, 4030, 4040])  
  print(zc, " ")  
end
```

```
areas = Dict("Center" => 4020,  
            "South"   => 4030,  
            "North"   => 4040)
```

```
for (area, zc) in areas  
  println(area, ": ", zc)  
end
```

For a dictionary, destructuring (key, value) makes the role of each item explicit.

Nested loops can be written in two equivalent forms

Explicit nesting

```
for i in 1:3
  for j in "hello"
    println(i, " ", j)
  end
end
```

Compact form

```
for i in 1:3, j in "hello"
  println(i, " ", j)
end
```

Both forms iterate over every pair (i,j) .

Julia offers several ways to write functions

The form should match the complexity and intended use of the computation.

- **One-line function:** compact definition for a simple expression.
- **Anonymous function:** useful when a function is needed locally, for example by `map`.
- **General function:** a named block containing several instructions, branches, or intermediate variables.

One-line functions are ideal for simple formulas

```
f(x) = x^2 + 7
```

```
f(2)      # 11
```

This compact syntax defines an ordinary named Julia function.

Anonymous functions are values without a function name

The syntax `x -> expression` creates a function.

```
(x -> x^2 + 7)(2)           # scalar call
```

```
map(x -> x^2 + 7, [2, 7, 4]) # apply to a vector
```

```
((x, b) -> x^2 + b)(2, 7)    # two arguments
```

Anonymous functions are especially useful when passing a short computation as an argument to another function.

General functions use a function ... end block

```
function affinefct(x)
    y = x^2 + 7
    return y
end
```

```
affinefct(2)
```

For short functions, the explicit `return` is optional because Julia returns the value of the last evaluated expression.

Functions can accept several parameters

```
function affinefct(x, b)
    return x^2 + b
end
```

```
affinefct(2, 7)
```

The arguments are matched by position in the call.

Type annotations define more specific methods

```
function affinefct(x::Int64, b::Int64)
    return x^2 + b
end
```

The annotation restricts this method to `Int64` arguments.

Julia often does **not** require type annotations for performance. Their main semantic role is dispatch: they specify which method applies to which argument types.

Optional positional arguments provide default values

```
function affinefct(x, b = 3)
    return x^2 + b
end
```

```
affinefct(2)          # uses b = 3
affinefct(2, 7)       # uses b = 7
```

The caller may omit an optional argument when the default value is appropriate.

A function may return one, several, or no useful values

```
return expression
```

```
return expression1, expression2
```

```
return nothing
```

Multiple returned values are represented by a tuple and can be destructured:

```
function minmax2(a, b)
    return min(a, b), max(a, b)
end
```

```
lo, hi = minmax2(7, 2)
```

Rebinding and mutation are different operations

Arguments bind local names to objects. Rebinding a local name does not change the caller's binding.

```
function rebind(x)
  x = 0
end
```

But a function can mutate a mutable object that was passed to it:

```
function clearfirst!(a)
  a[1] = 0
end
```

```
v = [1, 2, 3]
clearfirst!(v)           # v is now [0, 2, 3]
```

By convention, a function name ending in ! warns that mutation may occur.

A scalar method may not accept an array

Suppose the method is explicitly restricted to integers:

```
function add1(x::Int64)
    return x + 1
end
```

```
add1(1)           # 2
add1([1,2,3])     # MethodError
```

The vector does not match the declared argument type Int64.

Broadcasting applies a function elementwise

Two equivalent ways to broadcast `add1` over an array are:

```
a = [1, 2, 3]
```

```
broadcast(add1, a)
```

```
add1.(a)
```

Both produce

```
[2, 3, 4]
```

The dot syntax is a central Julia idiom for elementwise computation.

One generic function can have several methods

```
bro1(x::Int64)    = println("Int: ", x)
bro1(x::Float64) = println("Flt: ", x)
bro1(x::Bool)     = println("Bool: ", x)
bro1(x)           = println("Others: ", x)
```

These definitions share the same function name but have different method signatures.

Use `methods(bro1)` to inspect the available methods.

Dispatch selects the most specific applicable method

```
brl(8.3)           # Float64 method  
brl('c')           # fallback method  
brl(3)              # Int64 method on a 64-bit system  
brl(true)           # Bool method  
brl("hello")        # fallback method
```

Julia dispatches on the types of **all positional arguments**. This is why the mechanism is called **multiple dispatch**.

Types of Function Parameters in Julia

Parameter type	Example
Positional	<code>f(x)</code>
Typed	<code>f(x::Vector{Int})</code>
Optional positional	<code>f(x, order=:ascending)</code>
Keyword	<code>f(x; order=:ascending)</code>
Variable number	<code>f(x...)</code>

```
function sort_sequence( values::Vector{Int}; asc::Bool = true)
    if asc
        return sort(values)
    else
        return sort(values, rev = true)
    end
end
```

Function signature

`values` is a typed positional parameter, whereas `asc` is a typed keyword parameter with a default value.

Calling the Sorting Function

Use the default value

```
values = [7, 2, 9, 1, 5]
sort_sequence(values)
# [1, 2, 5, 7, 9]
```

Specify the keyword parameter

```
sort_sequence(values; asc = false)
# [9, 7, 5, 2, 1]
```

Invalid parameter types

```
sort_sequence([2.5, 1.5])
# MethodError
```

```
sort_sequence(values; asc = 1)
# TypeError
```

Role of the semicolon

Parameters after ; are passed by name. Their order therefore does not matter.

Common Built-in Types in Julia

Julia automatically determines the type of a value.

Category	Julia type	Example
Integer	Int64	x = 42
Floating-point number	Float64	x = 3.14
Boolean	Bool	x = true
Character	Char	x = 'A'
Character string	String	x = "Julia"
Complex number	Complex	x = 2 + 3im
Rational number	Rational	x = 2 // 3

```
typeof(42)           # Int64
```

```
typeof(3.14)         # Float64
```

```
n::Int = 10          y = Float64(n)           # Convert Int to Float64
```

Dynamic typing

A variable does not have a fixed type, but every Julia value has a well-defined concrete type.

Numerical Types and Result Types

The type of an arithmetic result depends on the operand types and the operation.

Expression	Result	Result type
5 + 2	7	Int64
5 + 2.0	7.0	Float64
5 / 2	2.5	Float64
div(5,2)	2	Int64
5 % 2	1	Int64

Inspecting and converting types

```
typeof(5 / 2)           # Float64
typeof(5 // 2)          # Rational{Int64}
Float64(5)              # 5.0
Int(3.8)                # InexactError
Int(round(3.8))          # 4
```

When different numerical types are combined, Julia generally promotes them to a common type that can represent both operands.

Creating and Combining Strings

```
language = "Julia"  
domain = "scientific computing"
```

Concatenation

```
text = language * " for " * domain  
# "Julia for scientific computing"
```

String interpolation

```
name = "Alice"  
score = 18
```

```
message = "$name obtained $score points"  
# "Alice obtained 18 points"
```

Repeating a string

```
repeat("Julia! ", 3)  
# "Julia! Julia! Julia! "
```

Main operators

Use `*` for concatenation and `$` to insert values into a string.

Common String Operations

```
s = "Julia programming"
length(s)           # 17
uppercase(s)        # "JULIA PROGRAMMING"
lowercase(s)        # "julia programming"
startswith(s, "Julia") # true
endswith(s, "ing")   # true
occursin("gram", s)  # true
```

Replacing text

```
replace(s, "Julia" => "Scientific")
# "Scientific programming"
```

Splitting and joining

```
words = split(s)
# ["Julia", "programming"]

join(words, "-")
# "Julia-programming"
```

A vector is a one-dimensional array

Arrays are ordered collections. A vector is an array with one dimension.

```
countries = ["Austria", "France", "Belgium"]  
primeNumbers = [2, 3, 5, 7, 11]  
mixed = [2, 3, 5, "seven", 11]
```

Julia uses **1-based indexing**: the first element of a vector has index 1, not 0.

A homogeneous literal such as `[2, 3, 5]` typically has a concrete element type, whereas mixed values may lead Julia to choose a more general element type.

Create an uninitialized typed vector

When the element type and size are already known, a vector can be allocated without initializing its entries.

```
v = Array{Int64}(undef, 5)
```

Equivalent, more explicit one-dimensional form:

```
v = Vector{Int64}(undef, 5)
```

`undef` means that the entries have not yet been initialized. They should be assigned before being used.

Creating Vectors in Julia

Vector creation

```
u = [3,1,0]
```

```
v = [0,3,1]
```

```
zeros(3)      # [0, 0, 0]
```

```
ones(3)       # [1, 1, 1]
```

```
collect(1:5)  # [1, 2, 3, 4, 5]
```

Accessing vector elements

```
u[1]          # 3
```

```
u[2:3]        # [1, 0]
```

```
length(u)     # 3
```

Basic operations

```
u + v         # [3, 4, 1]
```

```
u - v         # [3, -2, -1]
```

```
2 * u         # [6, 2, 0]
```

```
norm(u)       # sqrt(10)
```

Indexing

Julia uses one-based indexing: the first element of `u` is accessed with `u[1]`.

Products of Vectors in Julia

Let $u = [3, 1, 0]^\top$ and $v = [0, 3, 1]^\top$.

Elementwise product

```
u .* v  
# [0, 3, 0]
```

Dot product

$$u^\top v = 3(0) + 1(3) + 0(1) = 3$$

```
dot(u, v)      # 3  
u' * v         # 3
```

Cross product

$$u \times v = \begin{bmatrix} 1 \\ -3 \\ 9 \end{bmatrix}$$

```
cross(u, v)  
# [1, -3, 9]
```

Outer product

```
u * v'  
# 3 x 3 matrix
```

$$uv^\top = \begin{bmatrix} 0 & 9 & 3 \\ 0 & 3 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

Plotting Vector Operations

```
using LinearAlgebra
using Plots
```

```
u = [3.0, 0, 1.0]
v = [0, 3.0, 1.0]
```

```
vectors = [u, v, u + v, cross(u, v)]
names = ["u", "v", "u + v", "u x v"]
colors = [:blue, :red, :green, :purple]
```

```
p = plot3d(
    xlims=(-4, 10), ylims=(-1, 10),
    zlims=(-4, 10),
    xlabel="x", ylabel="y", zlabel="z",
    legend=:topleft
)
```

The four vectors start at the origin:

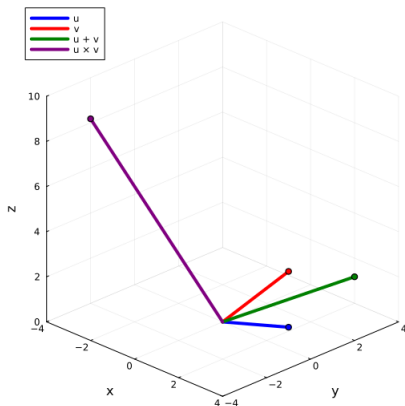
$$u = \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix}, \quad v = \begin{bmatrix} 0 \\ 3 \\ 1 \end{bmatrix},$$

$$u+v = \begin{bmatrix} 3 \\ 3 \\ 2 \end{bmatrix}, \quad u \times v = \begin{bmatrix} -3 \\ 6 \\ -3 \end{bmatrix}.$$

Geometric property

The cross product $u \times v$ is perpendicular to both u and v .

Visualization of Vector Operations



$$u = \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix}, \quad v = \begin{bmatrix} 0 \\ 3 \\ 1 \end{bmatrix}$$

$$u + v = \begin{bmatrix} 3 \\ 3 \\ 2 \end{bmatrix}$$

$$u \times v = \begin{bmatrix} -3 \\ -3 \\ 9 \end{bmatrix}$$

A matrix is a two-dimensional array

Spaces separate columns and semicolons separate rows in a matrix literal.

```
A = [1 2; 3 4]
T = ['a' 'b' 'c';
     'x' 'y' 'z']
```

Thus A is a 2×2 matrix and T is a 2×3 matrix.

More generally, Julia's Array type can represent data with any number of dimensions.

An array of arrays is different from a matrix

A collection can contain arrays as its elements. This is useful for irregular or “jagged” data.

```
A = [[1, 2], [3, 4, 6], [1]]
```

```
A[1]          # [1, 2]
```

```
A[2][3]       # 6
```

Here A is a vector whose elements are vectors of different lengths. It is **not** a rectangular matrix.

For rectangular numerical data, a true matrix is usually more natural and more efficient.

Allocate typed multidimensional arrays

Specify the element type followed by the dimensions.

```
A = Array{Int64}(undef, 2, 3)
B = Matrix{Int64}(undef, 2, 3)
```

Initialized multidimensional arrays are just as easy to create:

```
Z = zeros(Float64, 2, 3, 4)
O = ones(Float64, 2, 3, 4)
P = fill(pi, 2, 3)
```

`Matrix{T}` is an alias for a two-dimensional `Array{T,2}`.

Index multidimensional arrays by dimension

For an ordinary multidimensional array, provide one index per dimension.

`m[2, 1]`

`T[:, 2]`

The colon `:` selects all valid indices in that dimension, so `T[:, 2]` selects the entire second column.

For an array of arrays, indexing happens successively:

`A[2][3]`

Inspect array dimensions with size and length

```
size(T)           # dimensions as a tuple  
size(T, 2)        # size of the second dimension  
length(T)         # total number of elements
```

For a 2×3 matrix:

`size(T) = (2,3),` `size(T,2) = 3,` `length(T) = 6.`

Commas and spaces produce different shapes

These two literals contain the same numbers but have different dimensions:

```
v = [2, 3, 5, 7]      # Vector: 4 elements  
r = [2 3 5 7]         # Matrix: 1 x 4
```

Use `reshape` to reinterpret the same elements with a new shape:

```
reshape(r, 4)  
reshape(r, (2, 2))
```

The total number of elements must remain unchanged.

Assignment does not copy a mutable array

Consider:

```
g = [2, 3, 4, 5]
```

```
h = g
```

```
g[2] = 0
```

```
println(h)           # [2, 0, 4, 5]
```

`g` and `h` refer to the **same mutable array**. Mutating it through one variable is therefore visible through the other.

This aliasing behavior is important whenever arrays are passed around a program.

Choose the appropriate copying operation

Julia provides several related operations:

Operation	Purpose
<code>copy(A)</code>	create a shallow copy of A
<code>deepcopy(A)</code>	recursively copy nested mutable objects
<code>similar(A)</code>	allocate an uninitialized array with similar type and shape

```
B = copy(A)
```

```
C = deepcopy(A)
```

```
D = similar(A)
```

Comprehensions construct arrays concisely

A comprehension describes which expression to evaluate over one or more iterators.

```
[i for i in 1:5]  
[i + j for i in 1:2 for j in 1:4]  
[i + j for i in 1:2, j in 1:4]  
[i for i in 1:10 if i % 2 == 1]  
[exp(i) for i in 1:3]
```

Note the distinction: repeated for clauses produce a one-dimensional result, while comma-separated iterators produce a multidimensional array.

Use vectors as dynamic lists

Let $L = [1,2,3,4,5]$. Elements can be inserted at either end.

```
push!(L, 10)           # add at the end  
pushfirst!(L, 0)       # add at the front
```

`splice!` can replace or insert at a specified position:

```
splice!(L, 3, [L[3], 7, 4])
```

Because these operations mutate L , their names end in `!`.

Remove elements from a vector

Starting again with `L = [1,2,3,4,5]`:

```
pop!(L)           # remove and return the last element
popfirst!(L)      # remove and return the first element
splice!(L, 2)     # remove and return the element at index 2
```

These operations change both the contents and the length of the vector.

Matrices and Matrix Operations in Julia

Creating matrices

`using LinearAlgebra`

```
A = [1 2; 3 4]
```

```
B = [5 6; 7 8]
```

Basic operations

```
A + B
```

```
2 * A
```

```
A * B           # Matrix product
```

```
A .* B          # Elementwise
```

product

```
A .+ 2          # Add 2 to each
```

entry

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}, \quad A.*B = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$$

Common matrix operations

```
transpose(A)    # Transpose
```

```
A'              # Adjoint
```

```
det(A)          # Determinant
```

```
tr(A)           # Trace
```

```
rank(A)         # Rank
```

```
inv(A)          # Inverse
```

Solving a linear system

```
b = [5, 11]
```

```
x = A \ b       # Solve A*x = b
```

```
# x = [1.0, 2.0]
```

Tuples are ordered and immutable

A tuple is an ordered collection whose structure cannot be modified after creation.

```
countries = ("Austria", "France", "Belgium")  
values = (2, 3, 5, "seven", 11)
```

```
values[1]          # 2  
values[end]        # 11
```

Tuples use parentheses rather than square brackets and, like arrays, are indexed from 1.

A tuple cannot be changed in place

Compare a mutable vector with an immutable tuple:

```
vct = [2, 7]  
push!(vct, 3)      # valid
```

```
tpl = (2, 7)  
push!(tpl, 3)      # ERROR
```

Immutability makes tuples useful for small fixed collections, such as coordinates, dimensions, or multiple return values.

Dictionaries map keys to values

A dictionary stores key-value associations instead of positions alone.

```
code = Dict(  
    43 => "Austria",  
    32 => "Belgium",  
    33 => "France"  
)  
  
code[33]           # "France"
```

Keys are unique. Dictionary iteration order should generally not be treated as part of the logical meaning of the data.

Dictionaries are mutable

Start with an empty dictionary, then add or remove associations.

```
code = Dict()
```

```
code[49] = "Germany"
```

```
code[33] = "France"
```

```
pop!(code, 33)
```

The key is used to access the associated value; it does not represent a numeric position in the collection.

Iterate over dictionary keys or values

Julia exposes iterators over the keys and values of a dictionary.

```
keys(code)  
values(code)
```

They can be consumed directly by a loop or materialized when an actual array is required:

```
collect(keys(code))  
collect(values(code))
```

Sets store unique values

A Set represents a collection in which each value occurs at most once.

```
a = Set()
b = Set([1, 2, 2, 3, 4])

b                # contains 1, 2, 3, 4
push!(b, 5)
delete!(b, 2)
```

Julia sets are **mutable**. Their iteration order should not be used to represent an ordered sequence.

Set-like operations express relationships between collections

```
union([1, 2], [3, 4])  
intersect([1, 2, 3], [3, 4, 5])  
setdiff([1, 2, 3], [3, 4, 5])
```

These operations correspond to familiar mathematical ideas:

- `union`: elements appearing in either collection;
- `intersect`: elements common to both collections;
- `setdiff`: elements of the first collection not in the second.

Choose a data structure according to its semantics

Structure	Ordered	Mutable	Main idea
Array	yes	yes	indexed sequence / grid
Tuple	yes	no	fixed ordered collection
Dictionary	not by semantics	yes	key-value mapping
Set	no	yes	unique elements

Choose the structure that best expresses what the data *means*, not merely one that can store the values.

Exercise 1 : Testing and Visualizing Whether Two Vectors Are Collinear

Two vectors u and v are collinear if there exists $\lambda \in \mathbb{R}$ such that $u = \lambda v$.

Examples

```
are_collinear([1, 2, 3], [2, 4, 6])    # 1
are_collinear([1, 2, 3], [2, 4, 7])    # 0
are_collinear([1, 2], [-3, -6])        # 1
^^I
```

Numerical tolerance

For floating-point vectors, a tolerance is preferable to an exact equality test because computations may involve rounding errors.

Exercise 2 : Solving a square linear system $Ax=b$

Condition	Solutions
$\text{rank}(A) = n$	Unique solution $x=A \setminus b$
$\text{rank}(A) = \text{rank}([A \ b]) < n$	Infinitely many solutions, e.g $x=\text{pinv}(A)*b$
$\text{rank}(A) < \text{rank}([A \ b])$	No exact solution, least square approximation

- If A is invertible, $A \setminus b$ computes the unique solution.
- If the system has infinitely many solutions, $\text{pinv}(A) * b$ returns the solution with minimum Euclidean norm.
- If the system is inconsistent, the function returns the approximate least square solution $\text{pinv}(A) * b$.

Calling the function

```
result = solve_system(A, b)
```

```
println(result.status)
```

```
println(result.x)
```